

Microtask Queues: Promise & process.nextTick() - Complete Guide

Table of Contents

1. Understanding Microtask Queues
2. How to Add Callbacks to Queues
3. Experiments with Code Examples
4. Advanced Scenarios
5. Interview Questions & Answers
6. Common Pitfalls & Best Practices

Understanding Microtask Queues

What are Microtask Queues?

In the Node.js Event Loop, there are **two special queues** called **Microtask Queues**. They are:

1. **nextTick Queue** - for `process.nextTick()` callbacks
2. **Promise Queue** - for Promise `.then()`, `.catch()`, `.finally()` callbacks

MICROTASK QUEUES (Highest Priority!)	
1. nextTick Queue - <code>process.nextTick()</code>	← Runs FIRST
2. Promise Queue - <code>Promise.then()</code> - <code>Promise.catch()</code> - <code>Promise.finally()</code>	← Runs SECOND

Why are they called "Microtasks"?

- They are **small, quick tasks**
- They have **HIGHEST priority** in the event loop
- They run **between every phase** of the event loop
- They run **after every single callback** from other queues

How to Add Callbacks to Queues

Adding to nextTick Queue

```
// Syntax
process.nextTick(callbackFunction);

// Example
process.nextTick(() => {
  console.log("This runs in nextTick queue");
});
```

When callback is added:

- Immediately when `process.nextTick()` is executed
- Goes to the end of the nextTick queue

Adding to Promise Queue

```
// Syntax
Promise.resolve().then(callbackFunction);

// Example
Promise.resolve().then(() => {
  console.log("This runs in Promise queue");
});

// Also works with Promise chains
fetch(url)
  .then(response => response.json()) // Added to Promise queue
  .then(data => console.log(data))    // Added to Promise queue
  .catch(error => console.log(error)) // Added to Promise queue
```

When callback is added:

- When the Promise is resolved/rejected
- Goes to the end of the Promise queue

Experiments with Code Examples

Experiment 1: Synchronous Code vs nextTick

Goal: Understand that synchronous code runs FIRST, always!

```
// index.js
console.log("console.log 1");
process.nextTick(() => console.log("this is process.nextTick 1"));
console.log("console.log 2");
```

Output:

```
console.log 1
console.log 2
this is process.nextTick 1
```

Step-by-Step Execution:

Time 1ms:

```
| log(...1) | ← Executes immediately
```

Output: "console.log 1"

Time 2ms:

```
| process.nextTick(callback) | ← Adds callback to nextTick queue
```

nextTick Queue: [callback1]

Time 3ms:

```
| log(...2) | ← Executes immediately
```

Output: "console.log 2"

Time 4ms: Call stack is EMPTY, Event Loop starts
nextTick Queue: [callback1]

↓

Execute callback1

Output: "this is process.nextTick 1"

Key Learning:

All synchronous code runs BEFORE any asynchronous code, no matter what!

Experiment 2: nextTick vs Promise Priority

Goal: Understand priority between two microtask queues

```
// index.js
Promise.resolve().then(() => console.log("this is Promise.resolve 1"));
process.nextTick(() => console.log("this is process.nextTick 1"));
```

Output:

```
this is process.nextTick 1
this is Promise.resolve 1
```

Step-by-Step Execution:

Step 1: Line 1 executes
 Promise.resolve().then(...) → Adds callback to Promise Queue
 Promise Queue: [callback1]

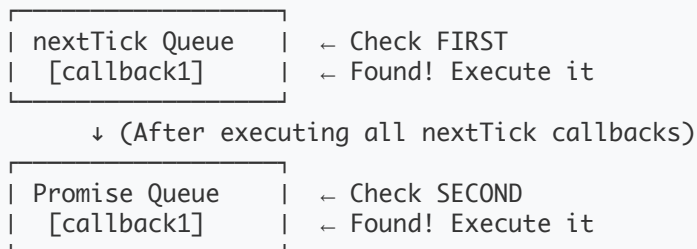
Step 2: Line 2 executes
 process.nextTick(...) → Adds callback to nextTick Queue
 nextTick Queue: [callback1]

Step 3: Call stack empty, Event Loop starts
 Priority Order:
 1. nextTick Queue (HIGHER priority)
 2. Promise Queue (LOWER priority)

Execution:
 1. Execute nextTick callback
 Output: "this is process.nextTick 1"
 2. Execute Promise callback
 Output: "this is Promise.resolve 1"

Visual Representation:

Event Loop Checking Queues:



Key Learning:

nextTick queue ALWAYS executes BEFORE Promise queue

Experiment 3: Multiple Callbacks in Both Queues

Goal: See the complete execution order with multiple callbacks

```
// index.js
console.log("Start");

process.nextTick(() => console.log("nextTick 1"));
process.nextTick(() => console.log("nextTick 2"));
process.nextTick(() => console.log("nextTick 3"));

Promise.resolve().then(() => console.log("Promise 1"));
Promise.resolve().then(() => console.log("Promise 2"));
Promise.resolve().then(() => console.log("Promise 3"));

console.log("End");
```

Output:

```

Start
End
nextTick 1
nextTick 2
nextTick 3
Promise 1
Promise 2
Promise 3

```

Key Learning:

All callbacks in nextTick queue execute first, then all callbacks in Promise queue

Advanced Scenarios

Scenario 1: Nested nextTick inside nextTick

This is the most tricky scenario!

```

process.nextTick(() => console.log("nextTick 1"));

process.nextTick(() => {
  console.log("nextTick 2");
  process.nextTick(() => {
    console.log("inner nextTick inside nextTick 2");
  });
});

process.nextTick(() => console.log("nextTick 3"));

Promise.resolve().then(() => console.log("Promise 1"));

```

Output:

```

nextTick 1
nextTick 2
nextTick 3
inner nextTick inside nextTick 2
Promise 1

```

Key Learning:

New nextTick callbacks added during execution are queued at the end and executed BEFORE moving to Promise queue

Callback are executed in Batches in Microtasks

Scenario 2: nextTick inside Promise callback

```
Promise.resolve().then(() => {
  console.log("Promise 1");
  process.nextTick(() => {
    console.log("nextTick inside Promise");
  });
});

Promise.resolve().then(() => console.log("Promise 2"));
```

Output:

```
Promise 1
Promise 2
nextTick inside Promise
```

Important Note: The nextTick callback is NOT executed immediately. It's added to the queue and will run after the current Promise queue is empty.

Key Learning:

When a nextTick is called inside a Promise callback, it executes AFTER all current Promise callbacks finish because it executes in batches

Scenario 3: Complex Mixed Example

This is an interview favorite!

```
console.log("Start");

setTimeout(() => console.log("Timer 1"), 0);

process.nextTick(() => console.log("nextTick 1"));
process.nextTick(() => {
  console.log("nextTick 2");
  process.nextTick(() =>
    console.log("inner nextTick")
  );
});
process.nextTick(() => console.log("nextTick 3"));

Promise.resolve().then(() => console.log("Promise 1"));
Promise.resolve().then(() => {
  console.log("Promise 2");
  process.nextTick(() =>
    console.log("nextTick inside Promise")
  );
});
Promise.resolve().then(() => console.log("Promise 3"));

console.log("End");
```

Output:

```

Start
End
nextTick 1
nextTick 2
nextTick 3
inner nextTick
Promise 1
Promise 2
Promise 3
nextTick inside Promise
Timer 1

```

Key Learning:

Microtask queues (nextTick + Promise) ALWAYS complete before moving to Timer/I/O/Check queues

Interview Questions & Answers

Q1: What is the difference between process.nextTick() and Promise.resolve().then()?

Answer:

Both add callbacks to microtask queues, but they have different priorities:

process.nextTick():

- Adds callback to the **nextTick queue**
- Has **HIGHEST priority** in the event loop
- Executes **before** Promise callbacks
- Should be used sparingly (can starve event loop)
- Use case: Critical operations that must run ASAP

Promise.resolve().then():

- Adds callback to the **Promise queue**
- Executes **after** all nextTick callbacks
- More standard way to handle async operations
- Better for general async work

Code Example:

```
Promise.resolve().then(() => console.log("Promise"));
process.nextTick(() => console.log("nextTick"));

// Output:
// nextTick
// Promise
```

Memory Trick:

- `nextTick` = "I need to run RIGHT NOW, next!"
- `Promise` = "I'll run soon, but nextTick goes first"

Q2: What is Microtask Queue Starvation? Give an example.

Answer:

Microtask Queue Starvation happens when microtasks keep adding more microtasks, preventing the event loop from moving to other queues (Timer, I/O, Check).

Dangerous Example:

```
function recursiveNextTick() {
  console.log("Running nextTick");
  process.nextTick(recursiveNextTick); // Adds itself again!
}

// This setTimeout will NEVER run!
setTimeout(() => {
  console.log("I will never execute!");
}, 0);

process.nextTick(recursiveNextTick);

// Output:
// Running nextTick
// Running nextTick
// Running nextTick
// ... (infinite loop!)
// setTimeout callback NEVER runs!
```

Why it happens:

Event Loop Flow:

1. Check nextTick Queue → Found callback
2. Execute callback → Adds another nextTick callback
3. Check nextTick Queue again → Found new callback
4. Execute callback → Adds another nextTick callback
5. Repeat forever...
6. NEVER reaches Timer Queue!

Real-world Problem:


```
// This can starve I/O operations
function processData(data) {
  if (data.length > 0) {
    console.log(data[0]);
    data.shift();
    process.nextTick(() => processData(data));
  }
}

const fs = require('fs');

// This file read might never complete!
fs.readFile('file.txt', (err, data) => {
  console.log("File read complete");
});

processData([1,2,3,4,5, /* ... thousands of items */]);
```

Solution:

```
// Use setImmediate instead!
function processData(data) {
  if (data.length > 0) {
    console.log(data[0]);
    data.shift();
    setImmediate(() => processData(data)); // Better!
  }
}

// Now I/O operations can run between iterations
```

Key Learning:

Never create recursive nextTick calls. Use setImmediate for deferred work.

Q3: When should you use process.nextTick()?**Answer:**

The official Node.js documentation recommends using `process.nextTick()` for only **two main reasons**:

1. To handle errors:

```
function doSomethingAsync(callback) {
  if (!callback) {
    // Handle error AFTER call stack unwinds
    process.nextTick(() => {
      throw new Error("Callback is required!");
    });
    return;
  }

  // Continue with async operation
  fs.readFile('file.txt', callback);
}
```

2. To allow callbacks to run after the call stack unwinds but before the event loop continues:

```
class EventEmitter {
  constructor() {
    this.events = {};
  }

  on(event, listener) {
    if (!this.events[event]) {
      this.events[event] = [];
    }
    this.events[event].push(listener);
  }

  emit(event, ...args) {
    if (this.events[event]) {
      // Use nextTick to ensure listeners run AFTER current code
      this.events[event].forEach(listener => {
        process.nextTick(() => listener(...args));
      });
    }
  }
}

const emitter = new EventEmitter();

// Set up listener
emitter.on('data', (data) => {
  console.log('Received:', data);
});

// Emit immediately
console.log('Before emit');
emitter.emit('data', 'Hello');
console.log('After emit');

// Output:
// Before emit
// After emit
// Received: Hello
```

When NOT to use process.nextTick():

```
// ❌ Don't use for general async work
process.nextTick(() => {
  // Regular async task
  doSomething();
});

// ✅ Use Promise or setImmediate instead
Promise.resolve().then(() => {
  doSomething();
});

// ✅ Or setImmediate for deferred work
setImmediate(() => {
  doSomething();
});
```

Key Learning:

Use process.nextTick() judiciously and only when you need the highest priority execution

Q4: Predict the output of this code:

```
console.log("1");

setTimeout(() => console.log("2"), 0);

Promise.resolve()
  .then(() => console.log("3"))
  .then(() => console.log("4"));

process.nextTick(() => console.log("5"));

console.log("6");
```

Answer:

Output:

```
1
6
5
3
4
2
```

Explanation:

STEP 1: Synchronous Code

Execute: `console.log("1")` → Output: "1"

Execute: `setTimeout` → Adds to Timer Queue

Execute: `Promise.resolve().then` → Adds to Promise Queue

Execute: `Promise.then` (chained) → Will add to Promise Queue when first resolves

Execute: `process.nextTick` → Adds to nextTick Queue

Execute: `console.log("6")` → Output: "6"

Current State:

nextTick Queue: [callback for "5"]

Promise Queue: [callback for "3"]

Timer Queue: [callback for "2"]

STEP 2: Event Loop - Microtask Queues

Process nextTick Queue:

Execute callback → Output: "5"

nextTick Queue: []

Process Promise Queue:

Execute callback → Output: "3"

This callback returns another `Promise.then()`

Adds new callback to Promise Queue

Promise Queue: [callback for "4"]

Execute callback → Output: "4"

Promise Queue: []

STEP 3: Event Loop - Timer Queue

Execute callback → Output: "2"

Timer Queue: []

Q5: What happens when you add a nextTick inside a Promise callback?

Answer:

The `nextTick` callback gets added to the `nextTick` queue, but it doesn't execute immediately. It waits until all current Promise callbacks finish.

Example:

```
Promise.resolve().then(() => {
  console.log("Promise 1");
  process.nextTick(() => console.log("nextTick inside Promise"));
});
```

```
Promise.resolve().then(() => console.log("Promise 2"));
```

```
// Output:
```

```
// Promise 1
```

```
// Promise 2
```

```
// nextTick inside Promise
```

Why?

Step 1: Execute first Promise callback
 Output: "Promise 1"
 Adds nextTick callback (but doesn't execute yet)
 nextTick Queue: [callback]

Step 2: Continue with Promise Queue
 Execute second Promise callback
 Output: "Promise 2"
 Promise Queue: [] ← Empty now

Step 3: Event Loop checks microtask queues again
 nextTick Queue: [callback] ← Found!
 Execute callback
 Output: "nextTick inside Promise"

Important Rule:

The event loop processes queues in phases. It finishes the current queue before checking other queues, even higher priority ones.

Exception: If you add a nextTick OUTSIDE the Promise chain while Promises are executing, it might execute between Promise callbacks:

```
Promise.resolve().then(() => {
  console.log("Promise 1");

  // Add nextTick from OUTSIDE this callback
  process.nextTick(() => console.log("nextTick"));
});

Promise.resolve().then(() => console.log("Promise 2"));

// The execution order depends on when the nextTick is added
// But generally: Promise 1, Promise 2, nextTick
```

Q6: Can microtasks block the event loop indefinitely?

Answer:

Yes! This is one of the biggest dangers of misusing `process.nextTick()`.

Example of blocking:

```

let counter = 0;

function infiniteNextTick() {
  counter++;
  console.log(`Iteration ${counter}`);
  process.nextTick(infiniteNextTick);
}

// This will NEVER run
setTimeout(() => {
  console.log("Timer callback - I'll never execute!");
}, 0);

// Start the infinite loop
process.nextTick(infiniteNextTick);

// The program will keep running infiniteNextTick forever
// Event loop NEVER moves to Timer queue
// Application becomes unresponsive

```

Why it happens:

Event Loop Trying to Progress:

1. Check nextTick Queue → Found callback
2. Execute callback → Adds another nextTick
3. Check nextTick Queue → Found callback (the one just added)
4. Execute callback → Adds another nextTick
5. Repeat forever...
6. NEVER moves to Timer Queue
7. NEVER moves to I/O Queue
8. Application is STUCK!

How to prevent:

1. **Never create recursive nextTick calls**
2. **Use setImmediate for recursive operations**
3. **Set limits on iterations**

Safe version:

```

let counter = 0;
const MAX_ITERATIONS = 100;

function safeRecursion() {
  counter++;
  console.log(`Iteration ${counter}`);

  if (counter < MAX_ITERATIONS) {
    setImmediate(safeRecursion); // Use setImmediate instead!
  }
}

setTimeout(() => {
  console.log("Timer callback - I'll execute!");
}, 0);

setImmediate(safeRecursion);

// Now the event loop can process other queues between iterations

```

Key Learning:

Microtasks can starve the event loop. Always use setImmediate for recursive deferred work.

Common Pitfalls & Best Practices

✗ Pitfall 1: Thinking nextTick executes "next"

```

// Many developers think this:
process.nextTick(() => console.log("This runs next tick"));
console.log("This runs current tick");

// They expect:
// This runs next tick
// This runs current tick

// But reality:
// This runs current tick
// This runs next tick

```

Why the confusion? The name `nextTick` is misleading. It doesn't mean "next tick of the event loop". It means "next, tick" as in "immediately next after current code".

❌ Pitfall 2: Not understanding Promise chaining

```
// Incorrect understanding:
Promise.resolve()
  .then(() => console.log("1"))
  .then(() => console.log("2"));

// They think both callbacks are added at once
// Reality: Second .then() callback is added AFTER first completes
```

Correct understanding:

```
Promise.resolve().then(() => {
  console.log("1");
  // Only NOW does the second .then() get added
});
```

❌ Pitfall 3: Mixing sync and async without understanding order

```
// Problematic code:
function getData() {
  if (cache.has(key)) {
    // Synchronous return
    return cache.get(key);
  } else {
    // Asynchronous return
    return fetchFromDB(key);
  }
}

// Better approach:
function getData() {
  if (cache.has(key)) {
    // Make it async too!
    return Promise.resolve(cache.get(key));
  } else {
    return fetchFromDB(key);
  }
}
```


✓ Best Practice 1: Use nextTick sparingly

```
// ✗ Avoid
process.nextTick(() => {
  doRegularWork();
});

// ✓ Prefer
Promise.resolve().then(() => {
  doRegularWork();
});

// ✓ Or for less critical work
setImmediate(() => {
  doRegularWork();
});
```

✓ Best Practice 2: Understand the priority order

Priority Order (High to Low):

1. Synchronous code (always first)
2. process.nextTick()
3. Promise callbacks
4. setTimeout/setInterval
5. setImmediate
6. I/O callbacks

✓ Best Practice 3: Avoid recursive nextTick

```
// ✗ Dangerous
function process() {
  doWork();
  process.nextTick(process); // Can starve event loop
}

// ✓ Safe
function process() {
  doWork();
  setImmediate(process); // Allows other queues to run
}
```

🎯 Quick Reference

Comparison Table

Feature	nextTick	Promise
Priority	Highest	Second highest
Syntax	<code>process.nextTick(cb)</code>	<code>Promise.resolve().then(cb)</code>

Feature	nextTick	Promise
Execution	Before Promises	After nextTick
Use case	Critical operations	General async work
Risk	Can starve event loop	Safer
Standard	Node.js specific	JavaScript standard

Execution Order Cheat Sheet

1. All synchronous code
↓
2. All nextTick callbacks
↓
3. All Promise callbacks
↓
4. Check: Any new nextTick/Promise? If yes, go to step 2
↓
5. Timer queue (setTimeout/setInterval)
↓
6. I/O queue
↓
7. setImmediate queue
↓
8. Close queue

Common Code Patterns

Pattern 1: Ensuring async consistency

```
function doAsync(callback) {
  if (cached) {
    process.nextTick(() => callback(cached));
  } else {
    fetchData((data) => callback(data));
  }
}
```

Pattern 2: Error handling

```
function doSomething(callback) {
  if (!callback) {
    process.nextTick(() => {
      throw new Error("Callback required");
    });
    return;
  }
  // ... rest of code
}
```

Pattern 3: Deferred execution

```
class MyClass {  
  constructor() {  
    // Defer initialization  
    process.nextTick(() => this.init());  
  }  
  
  init() {  
    // Initialization code  
  }  
}
```

Summary for Interviews

Key Points to Remember:

1. **Two microtask queues:** nextTick (higher) and Promise (lower)
 2. **Execution order:**
 - Synchronous code first
 - Then nextTick queue
 - Then Promise queue
 - Then other queues (Timer, I/O, Check, Close)
 3. **nextTick has highest priority** but can starve the event loop
 4. **Microtasks added during execution** are processed before moving to next queue
 5. **Use nextTick sparingly** - only for critical operations
 6. **Prefer Promises** for general async work
 7. **Never create recursive nextTick** - use setImmediate instead
 8. **Microtasks run between every phase** of the event loop
-